The Islamia University of Bahawalpur

Department of Computer Science

SOFTWARE REQUIREMENTS SPECIFICATION

(SRS DOCUMENT)

For

Smart University Bus Info System

Version 1.0

By

Muhammad Fahad

Roll No

F22BDOCS1M01107

Session Fall 2022 – 2026

Supervisor

Dr. Ibrahim

Revision History

Date	Description	Author	Comments
23 Nov 2025	Version 1	Muhammad Fahad	First Revision

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	Dr. Ibrahim	Supervisor, CSIT 21306	23 Nov 2025

Table of Contents

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to formally define the functional and non-functional requirements for the Smart University Bus Information System (SUBIS). This document provides a complete description of the system's features, operational behavior, constraints, and interactions with users and external components.
It serves as a contract between the development team and the stakeholders, including the project supervisor, students, transport administrators, and system designers.

The intended audience of this SRS includes:

- Project Supervisor
- Development Team
- Quality Assurance / Testers
- University Transport Department
- Future maintainers or researchers

1.2 Scope

The Smart University Bus Information System (SUBIS) is a real-time, GPS-enabled transportation information system designed specifically for university students and transport administrators.
The system provides dynamic bus tracking, estimated time of arrival (ETA), real-time bus routes, stop information, and crowdedness indicators. The system also leverages graph-based routing algorithms, mapping/localization technologies, and an LLM-based interface for user support.

The main objectives of SUBIS are:

- To provide students with accurate, up-to-date bus locations and ETAs.
- To improve transportation efficiency by analyzing route patterns, crowd levels, and flow.

- To reduce waiting time and uncertainty for students at bus stops.
- To provide transport administrators with an analytical dashboard to monitor routes, buses, and performance.
- To create an end-to-end system combining, backend services, and AI-powered interaction(optional).

SUBIS will **not** perform bus scheduling, driver management, or fare/payment processing. Its sole purpose is information collection, processing, and delivery.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SUBIS	Smart University Bus Information System
ETA	Estimated Time of Arrival
GPS	Global Positioning System
OSM	OpenStreetMap (open-source mapping dataset)
LLM	Large Language Model used for user queries
MERN	Technology stack: MongoDB, Express.js, React.js, Node.js
UI/UX	User Interface / User Experience
Real-time Updates	Data transmitted live using WebSocket or similar
Telemetry	Location and movement data received from users/buses
Crowd Estimation	Determining bus/stop crowdedness using user density or reports

---

1.4 References

The following documents and resources have been used while preparing this SRS:

1. **Project Proposal – Smart University Bus Information System (SUBIS)** (uploaded by student).
2. **SRS Document Template – Department of Computer Science**

3. **Software Design Document Guidelines (IEEE 1016)**
4. **Final Documentation Template** (provided by department).
5. **OpenStreetMap Official Documentation** – for referencing mapping APIs.
6. **IEEE Standard 830-1998** – Recommended practice for Software Requirements Specifications.

1.5 Overview

This SRS document is structured into three main chapters:

- **Chapter 1 – Introduction:**
  Provides the purpose, scope, definitions, references, and overview of the SRS.
- **Chapter 2 – Overall Description:**
  Presents a high-level understanding of the system, including product perspective, major system functions, user characteristics, constraints, and assumptions.
- **Chapter 3 – Specific Requirements:**
  Defines all functional and non-functional requirements in detail, including external interfaces, use cases, classes/objects, and constraints.

This document does **not** describe system design or implementation details. Those aspects are covered separately in the Software Design Document (SDD).

2. Overall Description

2.1 Product Perspective

The Smart University Bus Information System (SUBIS) is an independent software product designed to improve and optimize the transportation information system within the university environment. SUBIS replaces manual timetable checking, unreliable student-to-student communication, and traditional static bus schedules with a dynamic, real-time, intelligent solution.

The system integrates multiple components:

- A **web application** for students.
- A **backend server** for processing, storing, and distributing information.
- A **university admin dashboard** for route management and analytics.
- An **LLM-based assistant** for natural language queries (optional but included in scope).
- **Real-time GPS modules or mobile-based location tracking** for students and buses.

SUBIS communicates with external systems such as:

- OpenStreetMap (OSM) tile servers for map rendering.
- University email services for authentication.
- WebSocket (Socket.io) servers for real-time updates.

The system is **independent**, modular, and does not require integration with existing university ERP systems.

2.1.1 Operations

SUBIS will operate in the following modes:

- **Student Interactive Mode**
  Students open the web app to view bus locations, ETAs, stop information, and route suggestions. They may also enable GPS sharing for improved crowd prediction.
- **Real-Time Monitoring Mode**
  The backend continuously collects GPS data from students and buses, processes routes, updates ETAs, and pushes live updates to all connected devices.
- **Admin Operational Mode**
  Transport administrators can add/edit routes, analyze usage, check bus density, and monitor system health via the admin dashboard.
- **Unattended Operations**
  Even when no users are active, the backend continues logging telemetry, running analytics, and updating route statuses.
- **Backup & Recovery**
  Database snapshots and logs will be taken regularly. In case of downtime, the

system will recover from the latest saved state.

2.1.2 Site Adaptation Requirements

To deploy SUBIS, the following site-specific adaptations may be required:

- Availability of a **university-hosted server** or cloud environment (e.g., VPS, local machine, AWS EC2).
- Pre-loading official **university bus routes and stop coordinates** into the database.
- GPS-enabled devices for students.
- Stable Wi-Fi or mobile data coverage across the campus.

For Location HTML GPS API will be used.

2.2 Product Functions

SUBIS provides the following major system functions:

- **Real-Time Bus Tracking**
  Display current bus locations on an interactive map.
- **ETA Calculation**
  Estimate arrival time for a selected stop based on bus speed, distance, and route topology.
- **Route Visualization & Path Finding**
  Display available bus routes, stops, and required transfers if direct routes are unavailable.
- **Student GPS Location Collection (Optional)**
  Improve crowd estimation and route prediction by collecting user locations (opt-in).
- **Crowd Level Estimation**
  Show whether a bus or stop is crowded, moderate, or free using aggregated user data.
- **Admin Dashboard**
  Manage routes, buses, schedules, and view analytics (usage, peak times, crowd

heatmaps).

- **Notifications System**
  Send alerts for approaching buses, delays, and route changes.

- **LLM-based Query Assistance**
  Students can ask questions like “When will Bus 12 reach Gate #1?” or “Which bus should I take to Department X?”

These functions collectively provide an intelligent transportation information experience.

2.3 User Characteristics

Primary Users (University Students)

- Age: 17–28
- Basic smartphone and internet usage experience
- No technical background required
- Access via web interface

Secondary Users (Transport Administrators)

- Familiar with basic computer usage
- Require an analytic and management interface
- Responsible for updating routes, buses, schedules, and resolving issues

Tertiary Users (Supervisors / Auditors (Optional))

- May review system performance and reports
- Use the dashboard occasionally

The system must be designed to be intuitive, accessible, and easy to navigate for all users.

2.4 General Constraints

The system is constrained by the following factors:

- **Regulatory / Institutional Policies**
  GPS data must be collected only with consent and handled according to privacy guidelines.

- **Hardware Limitations**
  Students' devices may have low GPS accuracy; system must tolerate location noise.

- **Internet Connectivity**
  University areas with weak signals may cause temporary real-time disruptions.

- **Performance Constraints**
  Real-time updates must remain within 1–3 seconds of actual bus movement.

- **Mapping Engine Constraint**
  Must use open-source maps (OSM) to avoid licensing costs.

- **Security Constraints**
  Communication must be encrypted (HTTPS, WebSocket Secure).

- **Resource Constraints**
  University infrastructure may limit server hosting options.

2.5 Assumptions and Dependencies

The system relies on the following assumptions:

- Students will voluntarily share location data to improve ETA accuracy.

- Buses follow fixed or semi-fixed routes provided by the transport department.

- Internet connectivity is sufficient for real-time communication.

- Admins will keep route data updated and accurate.

- GPS delay and drift will remain within acceptable limits.

- Map tiles from OSM will remain publicly accessible.

- The backend server remains operational during transport hours.

Dependencies include:

- Node.js, Express.js, MongoDB backend stack
- Leaflet.js & OpenStreetMap
- University email verification or login system
- Socket.io for live communication
- Deployed hosting environment (local server or cloud)

3. Specific Requirements

3.1 External Interface Requirements

SUBIS interacts with multiple external systems, users, devices, and communication channels.
This section describes all interface requirements in detail.

3.1.1 System Interfaces

The system must interact with the following external systems:

- **University Email Authentication System**
  Used to verify student identity through university-issued email accounts.
- **OpenStreetMap (OSM) Tile Servers**
  Required for map rendering using Leaflet.js.
- **HTML5 Location API OR User Mobile Devices**
  Used to collect student or bus location data.
- **Optional Bus GPS Tracker Devices**
  If installed, trackers will send bus coordinates to the server at intervals.
- **Backend REST API + WebSocket Server**
  Provides data exchange between frontend and backend systems.

Functional Requirements:

- The system shall receive GPS coordinates every 1–5 seconds.
- The system shall provide APIs for retrieving current bus locations, routes, and stops.
- The system shall expose a WebSocket endpoint for real-time updates.

3.1.2 Interfaces (User Interfaces)

SUBIS provides the following user interfaces:

- **Student Mobile/Web App UI**
    - Real-time bus tracking screen
    - ETA display
    - Route selection
    - Stop view
    - Crowdedness indicator
    - Notifications page
    - Chatbot/LLM interface (optional)
- **Admin Dashboard UI**
    - Route management panel
    - Bus management panel
    - Analytics & statistics
    - Map overview of bus operations
    - User and telemetry logs

Usability Requirements:

- Interface must be responsive on mobile and desktop.
- Key information must be visible without extra clicks.

- Map interactions (zoom, pan, click) must be intuitive.

3.1.3 Hardware Interfaces

SUBIS does **not** require dedicated hardware but interfaces with:

- **Smartphones with GPS & internet** (Android/iOS)
- **Laptops/Desktop Computers** (admin interface)
- **Optional Bus GPS Trackers via HTML5 Location API**
    - Must be able to send latitude, longitude, speed, and timestamp.

If no hardware trackers exist, student devices can serve as the primary data providers.

3.1.4 Software Interfaces

SUBIS relies on the following software systems:

- **MongoDB** – Data storage
- **Node.js/Express.js** – Backend logic
- **React.js** – Frontend/mobile interface
- **Socket.io** – Real-time communication
- **Leaflet.js** – Interactive mapping
- **SMTP / OAuth** – Email verification
- **JSON Web Tokens (JWT)** – Authentication tokens

Data Format Requirements:

- All APIs shall use **JSON** format.
- GPS update packets must include:
  `{ userId/busId, latitude, longitude, speed, timestamp }`.

3.1.5 Communications Interfaces

- **HTTPS protocol** shall be used for all API calls.
- **WSS (Secure WebSocket)** shall be used for real-time tracking.
- **Latency requirement:**
  Update delay must not exceed **10 seconds** under normal load.
- **Network Reliability:**
  The system must handle intermittent connectivity gracefully.

3.2 Functional Requirements

Functional requirements are written as **The system shall…** statements with IDs for traceability.

✔ FR-01 User Authentication

- The system shall allow users to register/login using their university email.
- The system shall verify email ownership using OTP or verification link.
- The system shall allow password reset through university email.

✔ FR-02 User Location Sharing

- The system shall allow users to enable or disable location sharing.
- The system shall securely collect GPS coordinates when enabled.
- The system shall anonymize student locations for privacy.

✔ FR-03 Bus Real-Time Tracking

- The system shall display all active buses on a map.
- The system shall update bus locations in real-time.
- The system shall show the route path for each bus.

✔ FR-04 ETA Calculation

- The system shall calculate ETA using distance, speed, and route topology.
- The system shall update ETA whenever new location data is received.
- ETA must refresh at least every 10 seconds.

✔ FR-05 Route & Stop Management (Admin Only)

- Admin can add, edit, or delete routes.
- Admin can add or modify bus stops with coordinates.
- Admin can choose route order (from stop 1 to last stop).

✔ FR-06 Crowdedness Estimation

- The system shall estimate crowd level using:
    - Number of nearby student devices
    - Direct reports from students
- The system shall display crowd status as:
    - Low / Moderate / High

✔ FR-07 Notifications

- Students can subscribe to arrival alerts for selected stops.
- System shall push notifications for:
    - Bus nearing a stop
    - Delay alerts
    - Route changes

✔ FR-08 LLM Interface (Optional Feature)

- Students can ask natural language questions (e.g., “Where is Bus A12?”).
- System shall return text answers using predefined logic or an LLM backend.

✔ FR-09 Admin Dashboard

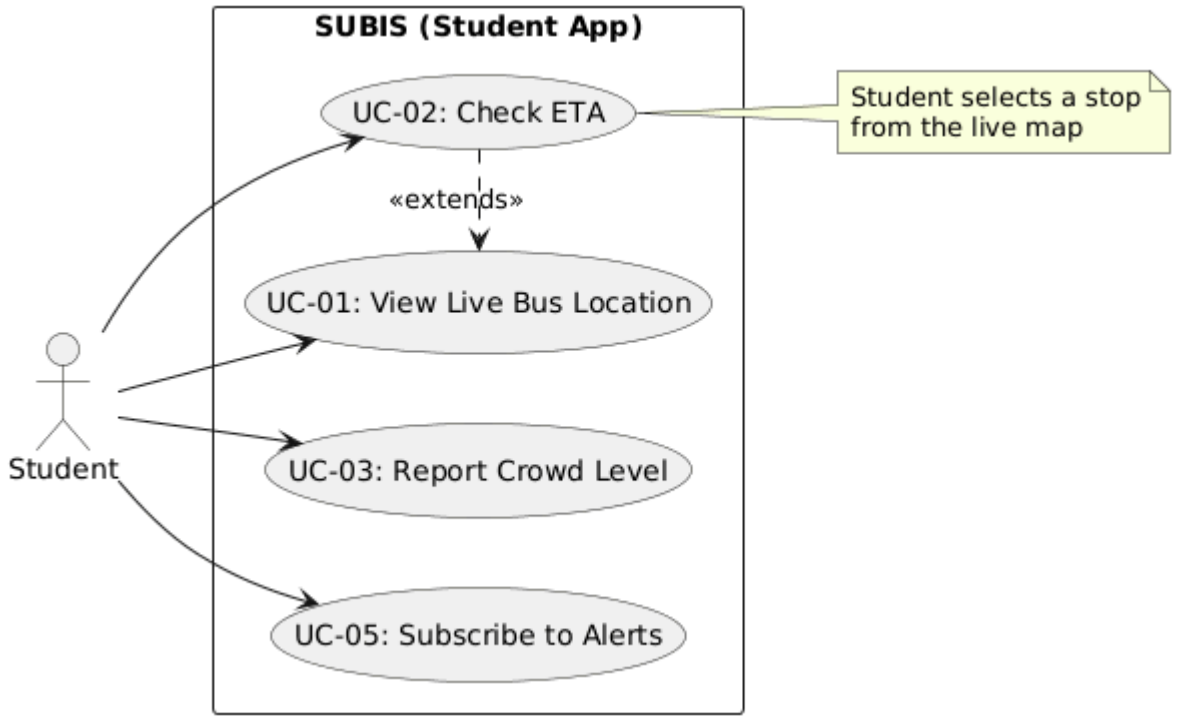
- Admin shall view real-time map of all buses.
- Admin shall view telemetry logs.
- Admin shall view daily/weekly usage analytics.

✔ FR-10 Data Logging & Export

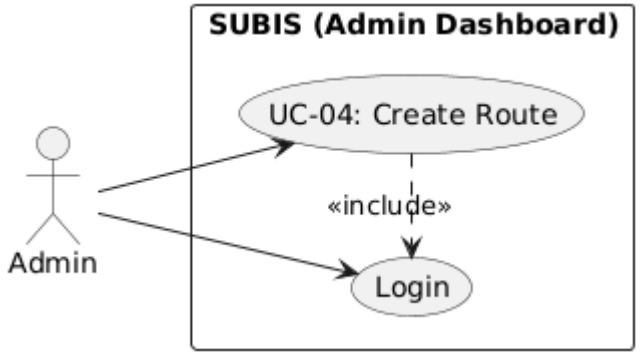
- System shall store telemetry for at least 90 days.
- Admin can export datasets (CSV/JSON).

3.3 Use Cases

Here are the primary use cases based on system functionality:



Description: This diagram illustrates the student's consumption-based interactions with the SUBIS application. The primary entry point is **View Live Bus Location (UC-01)**, which serves as the foundation for the interface. From the live map, students can extend their interaction to **Check ETA (UC-02)** for specific stops or **Subscribe to Alerts (UC-05)**. The diagram also highlights the **Report Crowd Level (UC-03)** feature, where students contribute data to the system either passively or actively.



Description: This diagram depicts the administrative management functions. The Admin is responsible for maintaining the system's static data. The core functionality shown is **Create Route (UC-04)**, which enables the definition of bus paths and stops. The diagram illustrates that **Authentication (Login)** is a mandatory prerequisite ("include" relationship) for accessing these sensitive configuration tools.

UC-01: View Live Bus Location

Actor: Student
Description: Student views live bus locations on map.
Trigger: User opens app.
Success: Buses appear with real-time updates.

UC-02: Check ETA

Actor: Student
Description: Student selects a stop to check arrival time.
Success: System displays ETA.

UC-03: Report Crowd Level

Actor: Student
Description: The crowd estimation is done via clustering algorithms to mark a stop “crowded” or “not crowded”.
Success: Data is aggregated and displayed.

UC-04: Admin Creates Route

Actor: Admin
Description: Admin creates a new route with stops.
Success: Route saved and visible in system.

UC-05: Subscribe to Alerts

Actor: Student
Description: Student subscribes to stop arrival notifications.
Success: System sends alerts.

3.4 Classes / Objects

Class: User

Attributes:

- userId, name, email, role, locationOptIn
  Methods:

- login(), updateLocation(), subscribeToAlerts()

Class: Bus

Attributes:

- busId, routeId, currentLocation, speed, occupancy
  Methods:

- updateTelemetry(), reportStatus()

Class: Route

Attributes:

- routeId, name, stops[], polyline
  Methods:

- addStop(), updateRoute()

Class: Stop

Attributes:

- stopId, name, coordinates, waitingCount
  Methods:

- updateCrowdStatus()

Class: Telemetry

Attributes:

- telemetryId, sourceId, timestamp, lat, long, speed
  Methods:

- process(), store()

3.5 Non-Functional Requirements

3.5.1 Performance

- 95% of location updates must be processed within **500 ms**.
- ETA must refresh at least every **10 seconds**.

3.5.2 Reliability

- System uptime must be at least **99%** during operation hours.

3.5.3 Availability

- System must support peak usage during university start/end times.

3.5.4 Security

- All communication must use HTTPS/WSS.
- Sensitive data must be encrypted at rest.
- No location data stored without user consent.

3.5.5 Maintainability

- Codebase must follow modular architecture.
- APIs must be versioned.

3.5.6 Portability

- System must run on Linux servers and major cloud providers.
- Frontend must run on all modern browsers and smartphones.

3.6 Inverse Requirements

- The system shall **not** track users when location sharing is disabled.
- The system shall **not** collect any personally identifiable GPS logs without consent.
- The system shall **not** manage bus scheduling or driver attendance.

3.7 Logical Database Requirements

Entities include:

- User(userId, name, email, role, location)
- Bus(busId, routeId, location, speed)
- Route(routeId, name, stops[])
- Stop(stopId, name, coordinates)
- Telemetry(id, sourceId, timestamp, location)

Relationships:

- Route → Stops (1:N)

- Route → Buses (1:N)
- User → Telemetry (1:N)

3.8 Design Constraints

3.8.1 Standards Compliance

- Must adhere to IEEE SRS Standards.
- Must use open-source mapping tools (OSM).
- Must follow university privacy guidelines.